

DSA5208 可扩展分布式数据科学计算

MongoDB 可调一致性与 客户端中心一致性实验

项目一 · 完整实验报告 (中文版)

作者 : Pan Yuxiang、Hou Jiacheng、Wu Jiarui

提交日期 : 2026 年 9 月 27 日

实验平台 : Google Cloud Compute Engine (新加坡)

核心结论 : 因果会话配合 majority 读写在成功操作中保持了四项保证 ;
无因果会话的 w:1 + local + Secondary 读取在受控复制延迟下暴露了全部三类读因果异常。

Repository: github.com/Panyuxiang0830/DSA5208_project1

目录

目录	2
摘要	3
1. 项目目标与研究问题	3
2. 背景：四项客户端中心一致性	3
3. 数据库与部署架构	3
3.1 软件与云资源	3
3.2 安装与初始化	4
4. 一致性配置与预测	4
5. 实验设计	5
5.1 工作负载与测量	5
5.2 故障注入与恢复	5
6. 结果	6
6.1 S0-S3：稳态矩阵	6
6.2 延迟	6
6.3 T1/T2：选举过渡窗口	7
6.4 S4：受控复制延迟	8
7. 预测与观察对照	8
8. 局限性与有效性威胁	8
9. 可复现性	9
9.1 主要命令	9
9.2 正式运行标识	9
10. 结论	10
参考文献	10
AI 使用声明	10

摘要

本项目在单台 Google Cloud 虚拟机中使用 Docker Compose 部署三节点 MongoDB 8.0.29 副本集，并系统研究读关注级别、写关注级别、读偏好与因果一致会话如何影响应用观察到的四项客户端中心一致性保证：读己之写、单调读、单调写和写跟随读。实验由正常运行、Secondary 停机、Primary 停机、Primary 网络分区、选举过渡窗口以及受控复制延迟六类场景组成。正式基线与稳态故障矩阵对每个配置和模型运行 500 个序列及 3 个随机种子；过渡窗口持续 35 秒；复制延迟实验冻结一个可读 Secondary 的应用线程。结果显示，C1、C2 与 C4 在所有成功检查中均未出现一致性违例；弱配置 C3 在正常状态的读己之写违例率为 76.07%，在 S1-S3 为 64.60%-69.93%。选举窗口产生短暂操作错误，但没有在强配置中引入新的成功读异常。S4 将弱配置的读己之写、单调读和写跟随读违例率分别放大到 83.33%、49.80% 和 50.00%，而单调写仍未违例。结果支持 MongoDB 文档对因果一致会话与 majority 读写关注级别的描述，同时说明“未观察到违例”是特定实验负载下的证据，而非对所有执行历史的形式证明。

关键词：MongoDB；副本集；可调一致性；因果一致性；故障注入；复制延迟

1. 项目目标与研究问题

课程任务要求搭建一个支持可调一致性的复制型分布式数据库，在正常运行、节点故障和网络分区下预测并检验应用观察到的客户端中心一致性。本项目选择 MongoDB，因为其驱动层同时暴露会话、读关注、写关注和读偏好，能够把抽象一致性模型映射为可复现的客户端配置。

- 不同配置是否保持读己之写 (RYW)、单调读 (MR)、单调写 (MW) 与写跟随读 (WFR) ？
- 稳定故障状态与选举过渡窗口分别如何影响一致性和可用性？
- 受控复制延迟能否把偶发的旧读放大为稳定、可解释的异常？

2. 背景：四项客户端中心一致性

四项保证是同一因果一致性框架中的并列会话保证，并不是简单的单轴强弱排序。一个系统可能满足其中一项而不满足另一项；因果一致性通常要求四项共同成立。下表给出本项目的可执行判定。

模型	应用期望	实验判定
RYW	客户端写入后应读到该写或更新版本。	observed_version < acknowledged_version
MR	同一客户端后续读取不能回到更旧版本。	later_version < earlier_version
MW	同一客户端的写入按发出顺序生效。	acknowledged history/final version out of order
WFR	读取后发出的依赖写必须建立在所读版本之上。	parent missing/stale predicate/dependent write missing

3. 数据库与部署架构

3.1 软件与云资源

项目项	采用配置
云平台	Google Cloud Compute Engine；项目 nifty-pursuit-505911-n0
虚拟机	dsa5208-mongodb；asia-southeast1-b (Singapore)；e2-standard-2；2 vCPU；8 GB RAM
系统	Ubuntu 24.04.4 LTS；Linux 6.17.0-1022-gcp
存储	30 GB balanced persistent disk；MongoDB named volumes
容器	Docker Engine 29.7.2；Docker Compose v5.5.0
数据库	MongoDB Community 8.0.29；three data-bearing voting members；no arbiter
客户端	Python 3.12-slim；PyMongo 4.17.0
资源限制	每个 mongod 容器 1536 MB RAM；WiredTiger cache 0.5 GB

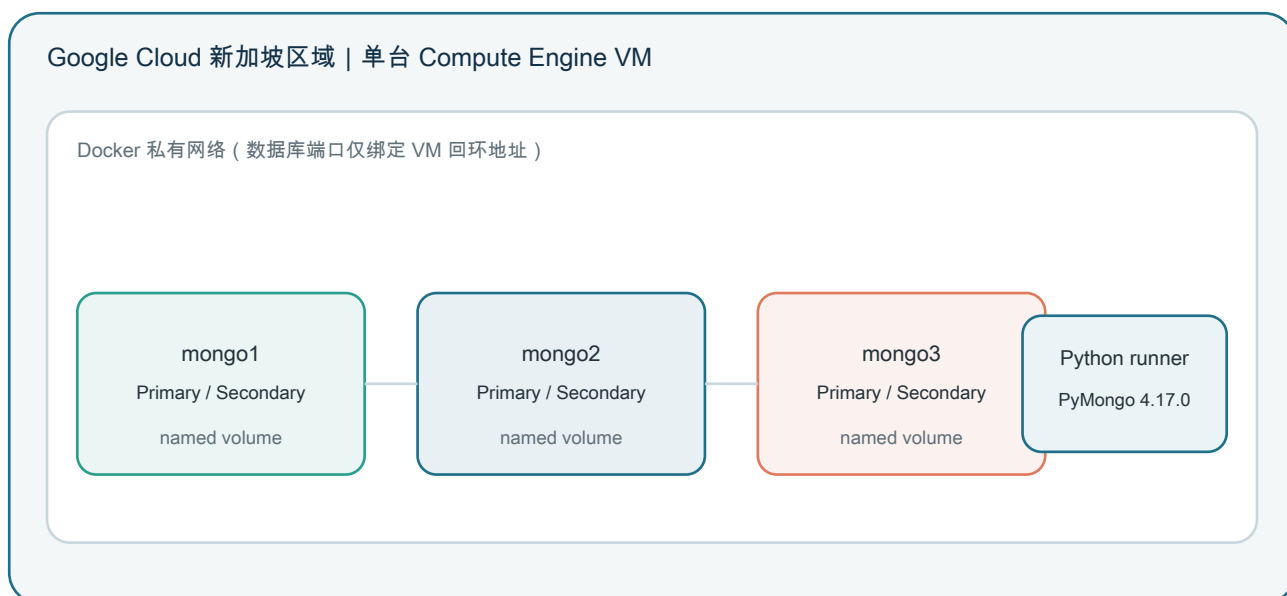


图 1. 实验部署架构。三个 MongoDB 容器共享一台 VM，但具有独立容器、进程和持久卷。

该方案满足“复制型分布式数据库”的逻辑部署要求，并便于确定性注入容器停机与容器网络分区。数据库端口仅绑定虚拟机回环地址，外部网络不直接暴露 MongoDB。其主要代价是节点共享宿主机、磁盘与底层网络，因此不能代表跨主机故障独立性。

3.2 安装与初始化

```

git clone https://github.com/Panyuxiang0830/DSA5208_project1.git
cd DSA5208_project1
./scripts/bootstrap-ubuntu.sh
cp .env.example .env
docker compose pull
docker compose up -d --wait mongo1 mongo2 mongo3
docker compose run --rm mongo-init
./scripts/cluster_status.sh
docker compose build runner
docker compose run --rm --no-deps runner
  
```

初始化脚本创建三名投票且存储数据的副本集成员。正常状态下一个节点为 Primary、两个节点为 Secondary；选举由 MongoDB 副本集协议负责。容器删除或重启不会删除 named volume，因此数据库文件在常规恢复流程中保留。

4. 一致性配置与预测

MongoDB 的 write concern 决定写入在何种复制确认条件下返回；read concern 决定读取允许观察的数据可见性；read preference 决定读取路由到 Primary 还是 Secondary；因果会话则由驱动携带因果时间信息。MongoDB 文档说明，在因果一致会话中配合 majority 读写关注，可以跨成员提供 RYW、MR、MW 与 WFR。

配置	会话	写关注	读关注	读偏好	预测
C1	因果	majority	majority	primary	四项均成立
C2	因果	majority	majority	secondary	四项均成立；可能等待副本
C3	无因果	w: 1	local	secondaryPreferred / Secondary	RYW、MR、WFR 可能违例；MW 通常成立
C4	无因果	majority	majority	primary	本负载中预期四项成立；不等同通用因果会话

C3 是主动选择的弱配置：w:1 只要求单个 mongod 确认，local 读取不保证数据已被多数节点提交，而 Secondary 读取可能落后于 Primary。C4 用于分离“majority + Primary”与“因果会话”各自的作用；它在本实验的单客户端、Primary 定向负载中很强，但不应外推为所有跨客户端因果历史的保证。

5. 实验设计

5.1 工作负载与测量

每个测试文档包含单调递增的版本号、写入者/运行标识和依赖字段。运行器记录每次数据库操作的开始时间、结束时间、结果、错误、读取成员与模型判定，并输出原始 JSONL、汇总 JSON 和 Markdown 摘要。每次正式基线/稳态故障实验对每个配置-模型组合运行 500 个序列，使用随机种子 20260830、20260831 与 20260832，共 1,500 次主要检查。所有数据库操作错误与一致性违例分开统计：错误表示请求未成功完成，违例只针对可判定的成功历史。

场景	故障/控制	测量目的
S0	无故障	建立正常运行基线
S1	停止一个 Secondary	验证多数派仍存活时的稳态行为
S2	停止原 Primary，等待选举完成	选举后两节点拓扑
S3	将原 Primary 与多数派及客户端断开	Primary 侧网络分区后的稳态
T1	停止 Primary 时持续发送 RYW	测量选举窗口的一致性与可用性
T2	分区 Primary 时持续发送 RYW	测量网络分区选举窗口
S4	冻结一个可读 Secondary 的复制应用	放大可解释的旧读

5.2 故障注入与恢复

S1 与 S2 使用 Docker 停止指定角色的容器。S3 使用 docker network disconnect 将故障前 Primary 从副本集网络断开，使剩余两节点形成多数派并选出新 Primary。T1/T2 在故障前启动四种配置的并发循环，把请求按开始时间标注为 pre、election 或 post。S4 仅在测试专用 compose 覆盖中启用 enableTestCommands=1，并对一个 Secondary 执行 rsSyncApplyStop；完成后执行 rsSyncApplyStop 的逆操作、等待追平，并重新创建普通容器。每个故障场景结束后均运行 restore_cluster.sh 和 cluster_status.sh 验证三节点健康。

设计理由：S0-S3 区分稳态一致性，T1/T2 捕获常被“等待选举完成”掩盖的短暂不可用窗口，S4 则提供可控因果机制，以验证 C3 的异常确实来自副本落后而非测试噪声。

6. 结果

6.1 S0-S3：稳态矩阵

S0 共完成 60,012 次数据库操作、18,012 次一致性检查，错误为 0。S1、S2、S3 各完成 18,012 次检查且数据库操作错误均为 0。C1、C2、C4 在全部场景和四个模型中均为零违例；C3 的稳定异常只出现在 RYW。

场景	C3 RYW	违例率	其他 C3 模型	C1/C2/C4
S0	1,141 / 1,500	76.07%	MR 109/1,500 (7.27%); MW 0; WFR 0	0
S1	1,035 / 1,500	69.00%	MR 0; MW 0; WFR 0	0
S2	969 / 1,500	64.60%	MR 0; MW 0; WFR 0	0
S3	1,049 / 1,500	69.93%	MR 0; MW 0; WFR 0	0

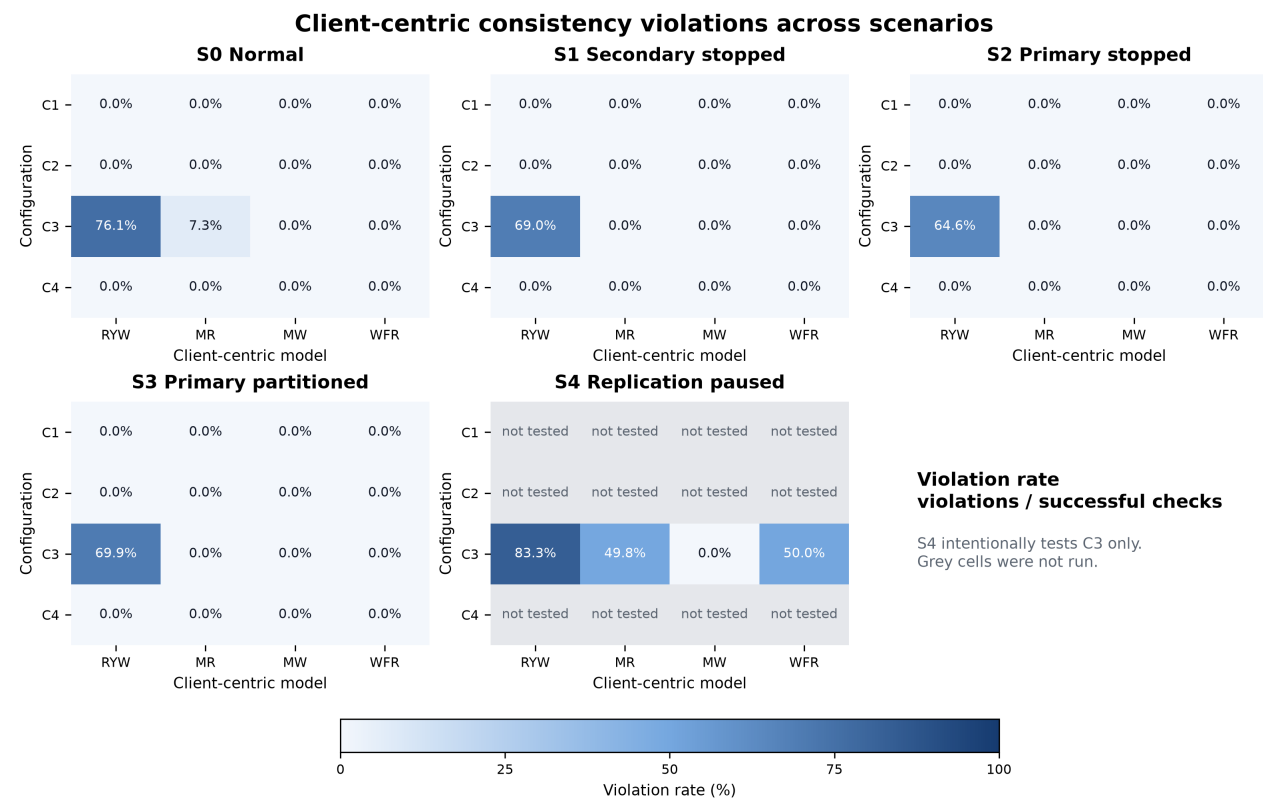


图 2. S0-S4 的一致性违例矩阵。颜色表示每 1,500 次主要检查的违例率。

S1-S3 的 C3 RYW 率略低于 S0，不代表故障提高了弱配置的一致性。副本数量、被选择的 Secondary 和运行时调度改变了旧副本被抽中的概率；这些率描述的是该部署中的经验频率，而不是协议保证。

6.2 延迟

配置	S0 p50 / p95	S1 p50 / p95	S2 p50 / p95	S3 p50 / p95
C1	3.19 / 9.24 ms	2.09 / 5.84	2.23 / 7.09	2.05 / 6.07
C2	3.43 / 9.11 ms	2.21 / 6.62	2.84 / 9.73	2.15 / 7.04
C3	1.48 / 5.00 ms	1.01 / 3.35	1.07 / 3.14	0.88 / 2.85
C4	3.22 / 9.13 ms	2.25 / 6.79	2.24 / 6.65	2.17 / 6.45

表中为 RYW 单次“写后读”对的端到端延迟。C3 延迟最低，与 w:1/local 不等待多数确认相符；这也是其更弱可见性的代价。C2 的 WFR 基线 p95 为 42.27 ms，高于 C1 的 19.37 ms，显示跨 Secondary 因果读取可能等待满足 afterClusterTime。不同故障场景的较低中位数受容器、缓存和运行时波动影响，不应解释为故障优化。

6.3 T1/T2：选举过渡窗口

场景	检查	违例	操作错误	平均选举时间	强配置成功操作
T1	9,635	2,420 (all C3)	34 (all election)	11,334 ms	0 violations
T2	10,974	2,532 (all C3)	60 (all election)	15,041 ms	0 violations

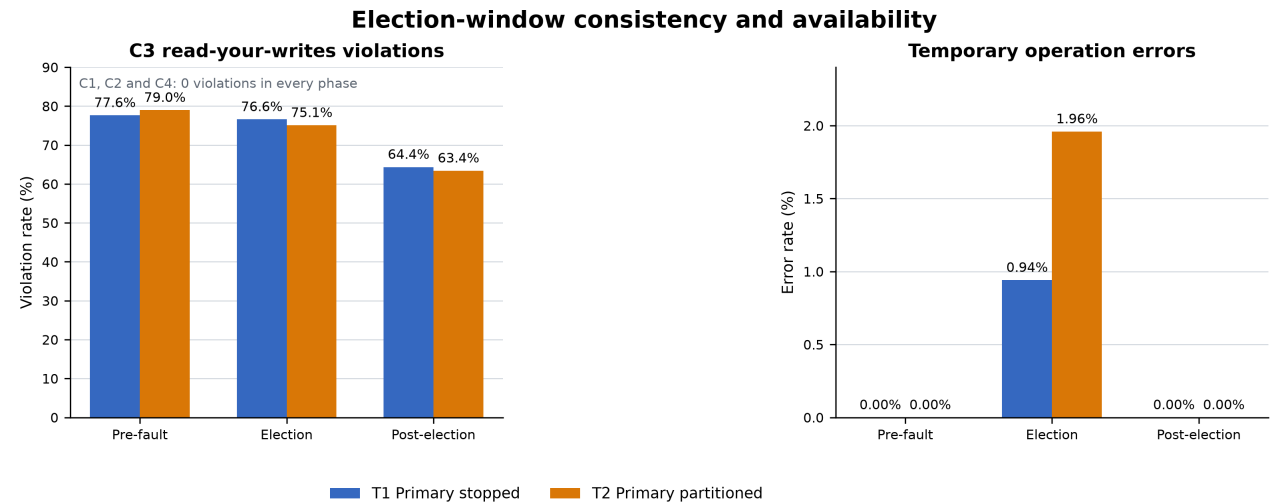


图 3. T1/T2 中各配置的成功检查、违例和操作错误。

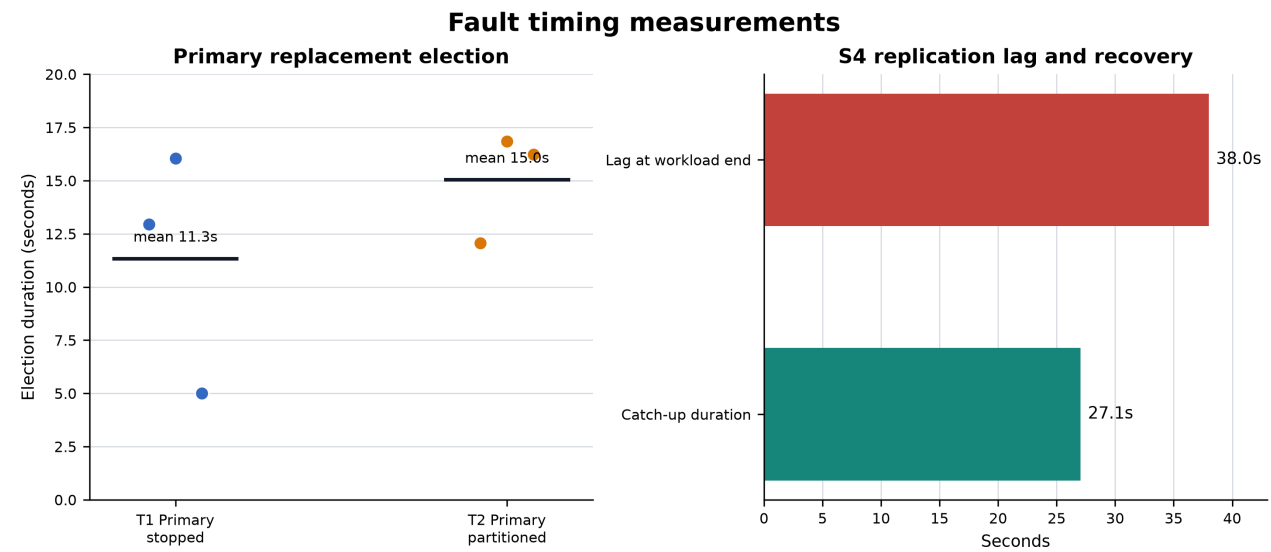


图 4. S2、S3、T1 与 T2 的选举时间。T1/T2 显示三个随机种子。

场景	pre	election	post
T1 C3	771/993 = 77.64%	537/701 = 76.60%	1,112/1,728 = 64.35%
T2 C3	849/1,075 = 78.98%	413/550 = 75.09%	1,270/2,003 = 63.40%

过渡实验给出两个互补结论。第一，C1、C2、C4 的成功 RYW 对仍然零违例；第二，选举阶段出现 34 与 60 次操作错误，说明强一致设置在无法立即找到可写 Primary 时牺牲了短暂可用性。C3 在故障前已经很弱，其 pre 与 election 违例率接近，因此选举没有创造一种新的 C3 一致性缺陷，主要新增的是暂时失败。

6.4 S4：受控复制延迟

模型	违例 / 检查	违例率	解释
RYW	1,250 / 1,500	83.33%	写后命中被冻结 Secondary，读到旧版本。
MR	747 / 1,500	49.80%	交替访问当前与冻结 Secondary，后读倒退。
MW	0 / 3 validations	0%	Primary 接收顺序写；最终版本有序。
WFR	750 / 1,500	50.00%	依赖写基于旧 Secondary 的父版本。

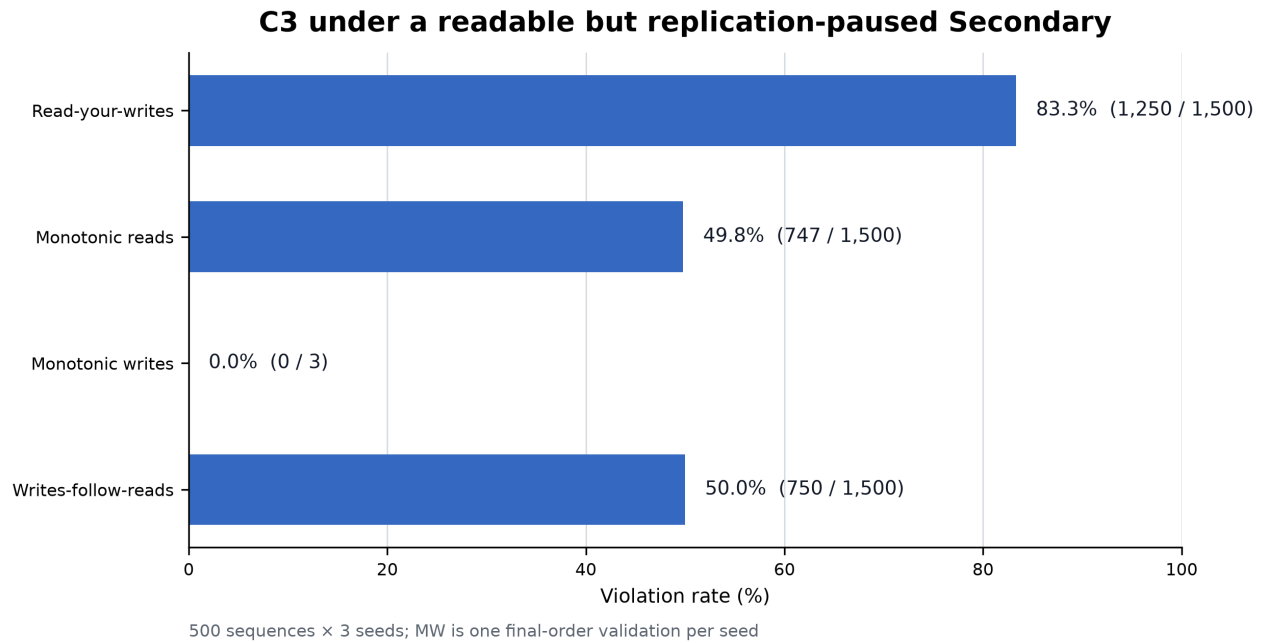


图 5. S4 中弱配置 C3 的各模型违例率。

S4 共 4,503 次检查、0 次数据库操作错误、2,747 次违例 (61.00%)。冻结节点在测试结束时落后 38 秒；恢复复制后，连续三次状态检查均在 1 秒内追平，完整恢复过程耗时 27,051 ms。MR 与 WFR 接近 50% 与测试在“冻结/当前”两个 Secondary 之间交替选择相吻合，提供了强机制证据。

7. 预测与观察对照

配置	预测	观察	结论
C1	四项成立	所有 S0-S3 检查与 T1/T2 成功 RYW 均零违例	一致
C2	跨副本因果会话仍成立	读取 Secondary 时仍零违例；部分尾延迟较高	一致
C3	RYW/MR/WFR 可能违例，MW 通常成立	S0 观察 RYW/MR；S4 观察 RYW/MR/WFR；MW 为零	一致；S4 补足机制
C4	本负载中成立，但不是通用因果保证	本实验零违例	有限支持

最重要的区分是安全性与可用性。强配置没有把选举期间的失败请求伪装成一致性违例：它们拒绝或重试无法安全完成的操作，而成功完成的历史继续满足 RYW。相反，C3 提供低延迟和较高可用读取，但允许成功返回旧版本。

8. 局限性与有效性威胁

- 三个节点位于同一 VM，共享硬件、磁盘与底层网络；容器隔离不能替代三台物理或虚拟主机。
- S1-S3 在选举完成后运行完整矩阵；T1/T2 只在过渡窗口测试 RYW，其他三项模型尚未持续压测。

- S3 将旧 Primary 同时与副本和客户端隔离，并未模拟分区两侧各有客户端的 split-client 历史。
- S4 使用 enableTestCommands 和 rsSyncApplyStop，是为了因果诊断而设计的极端失效点，不代表真实网络延迟分布。
- 每项仅使用三个种子；工作负载、驱动服务器选择与本机调度会改变经验违例率。零违例不是数学证明。
- 阶段以操作开始时间分类；跨越选举边界的单次操作仍只属于一个阶段。
- MW 运行了每个配置每种子 500 次顺序写，但汇总中的主要判定分母是每种子的最终历史验证，因此表中显示 3 次验证。
- 云账单存在报告延迟。实验结束时 VM 已停止；控制台显示本月总成本 0.14 美元、抵扣 0.14 美元、净成本 0，50 美元额度剩余 49.64 美元（约 99%）。

9. 可复现性

9.1 主要命令

```
# Baseline S0
docker compose run --rm --no-deps runner python -m experiments.run_baseline \
  --iterations 500 --seeds 20260830,20260831,20260832 --label formal

# Stable fault scenarios (replace the scenario name for S2/S3)
./scripts/run_fault_scenario.sh S1-secondary-failure 500 \
  20260830,20260831,20260832 formal

# Transition windows; repeat for seeds 20260830-20260832
./scripts/run_transition_scenario.sh T1-primary-stop-transition 20260830 formal 35 3
./scripts/run_transition_scenario.sh T2-primary-partition-transition 20260830 formal 35 3

# Controlled replication lag
./scripts/run_replication_lag_scenario.sh 500 \
  20260830,20260831,20260832 formal

# Recovery and report figures
./scripts/restore_cluster.sh
./scripts/cluster_status.sh
python3 -m analysis.generate_report_figures
```

仓库保存 compose 配置、初始化脚本、故障注入脚本、Python 工作负载、正式 summary JSON、汇总说明和生成图表的代码。原始 JSONL 可由上述命令重新生成。为避免意外计费，实验结束后应在 Google Cloud 控制台停止 VM，并确认实例状态为 TERMINATED；持久磁盘仍可能持续计费。

9.2 正式运行标识

场景	Selected formal run ID / suffix
S0	baseline-s0-formal-20260830T154711Z-3c931c32
S1	s1-secondary-failure-formal-20260830T181846Z-8e657452
S2	s2-primary-failure-formal-20260830T182320Z-f604f5eb
S3	s3-primary-partition-formal-rerun-20260830T183455Z-59f3d378
S4	s4-secondary-replication-lag-formal-20260831T081523Z-85b1e3b3
T1	6cad906c; 867b572c; ae94f015
T2	0609280f; b9f2cd1b; ceb526e5

10. 结论

实验完成了从理论预测、复制集部署、可调配置、四模型工作负载到故障注入和机制验证的闭环。结果表明，MongoDB 的强会话配置在本实验所有成功历史中保持四项客户端中心保证；弱 Secondary 读取能以更低延迟返回，但在正常复制、选举前后以及受控滞后下会返回旧版本。节点停机与网络分区最直接的过渡影响是短暂不可用，而不是强配置成功操作的一致性退化。最终，S4 把 RYW、MR 与 WFR 的弱行为同时放大，并保留 MW，从而说明四项会话保证是并列且可独立观察的。

参考文献

- [1] MongoDB Manual, Read Isolation, Consistency, and Recency. <https://www.mongodb.com/docs/manual/core/read-isolation-consistency-recency/>
- [2] MongoDB Manual, Read Concern. <https://www.mongodb.com/docs/manual/reference/read-concern/>
- [3] MongoDB Manual, Write Concern. <https://www.mongodb.com/docs/manual/reference/write-concern/>
- [4] MongoDB Manual, Read Preference. <https://www.mongodb.com/docs/manual/core/read-preference/>
- [5] MongoDB Manual, Self-Managed Replica Set Configuration. <https://www.mongodb.com/docs/manual/reference/replica-configuration/>
- [6] MongoDB PyMongo Driver Documentation, Transactions and Causal Consistency. <https://www.mongodb.com/docs/languages/python/pymongo-driver/current/crud/transactions/#causal-consistency>
- [7] Docker Documentation, Docker Compose. <https://docs.docker.com/compose/>
- [8] Docker CLI Documentation, docker network disconnect. <https://docs.docker.com/reference/cli/docker/network/disconnect/>
- [9] Google Cloud Documentation, Compute Engine Instances. <https://cloud.google.com/compute/docs/instances>
- [10] DSA5208 Lecture 3: Replication and Consistency (course material supplied by the instructor).

以上在线文档最后访问日期：2026-08-31。

AI 使用说明

本项目使用 OpenAI ChatGPT/Codex 协助解释课程概念、规划实验、起草和审阅脚本、组织结果、生成图表与润色报告。所有云资源创建、命令执行、故障注入、结果文件检查、参考文献核对及最终结论均由项目成员监督和验证。报告中的实验数据来自仓库中保存的真实运行输出，不由 AI 虚构。项目成员对提交内容、实验解释和任何错误承担最终责任。